



EÖTVÖS LORÁND TUDOMÁNYEGYETEM

INFORMATIKAI KAR

PROGRAMOZÁI NYELVEK ÉS FORDÍTÓPROGRAMOK
TANSZÉK

Elsőrendű logika, mint algebrai elmélet formalizálása Agdában

Témavezető:

Kaposi Ambrus

egyetemi docens

Szerző:

Szilágyi Péter

programtervező informatikus MSc

Budapest, 2025

EÖTVÖS LORÁND TUDOMÁNYEGYETEM

INFORMATIKAI KAR

DIPLOMAMUNKA TÉMABEJELENTŐ

Hallgató adatai:

Név: Szilágyi Péter

Neptun kód: CA36OR

Képzési adatok:

Szak: programtervező informatikus, mesterképzés (MA/MSc)

Tagozat : Nappali

Belső témavezetővel rendelkezem

Témavezető neve: Kaposi Ambrus

munkahelyének neve, tanszéke: **ELTE IK, Programozási nyelvek és Fordítóprogramok Tanszék**

munkahelyének címe: **1117, Budapest, Pázmány Péter sétány 1/C.**

beosztás és iskolai végzettsége: **egyetemi docens**

A diplomamunka címe:

Elsőrendű logika, mint algebrai elmélet formalizálása Agdában

A diplomamunka témája:

A kutatás célja az elsőrendű logika szintaxisának és szemantikájának formalizálása az Agda bizonyító rendszerben. Ezt algebrai módon tesszük meg, tehát a nyelv egy algebrai elmélet, és a szintaxis az iniciális modell. Az elsőrendű logika, mint algebrai elmélet tartalmaz egyenlőségeket, de ezek egy kivételével nem számítási szabályok, hanem csak a helyettesítési kalkulusra vonatkoznak. Az egy kivétel egy egyediség szabály, amely azt mondja, hogy tetszőleges két bizonyítás egyenlő. Így az elsőrendű logika szintaxisa kvóciensek nélkül megadható Agdában, a bizonyítások a definicionálisan irreleváns Prop fajtaban vannak. Az így megadott szintaxis inicialitását bizonyítjuk (tetszőleges modellbe pontosan egy homomorfizmus megy). Megadjuk a Tarski szemantikát, a klasszikus szemantikát, a Kripke szemantikát. Esetlegesen teljesség-bizonyítás a Kripke szemantikára (a teljesség a logika negatív fragmensére vonatkozik). A dolgozatban összefoglaljuk az elsőrendű logika szokásos megadási módjait, és összehasonlítjuk az algebrai változattal.

Budapest, 2023. 12. 13.

Tartalomjegyzék

1. Bevezetés	3
1.1. A dolgozat célja	3
1.2. Elsőrendű logika Agdában	6
1.2.1. Logikai állítások, mint típusok	6
1.2.2. Az univerzális kvantor, mint egy függő függvény típus	7
1.2.3. Az egzisztenciális kvantor ($\exists$), mint a függő pár típus (Σ)	7
1.2.4. Logikai összekötők, mint meglévő típuskonstruktorok	7
1.2.5. Egyenlőség és elsőrendű termék	8
1.2.6. Példa bizonyítás	8
2. Logika	9
2.1. Bevezetés a logikába	9
2.1.1. Propozíciós (nulladrendű) logika	9
2.1.2. Predikátumlogika (elsőrendű logika)	10
2.1.3. A propozíciós és az elsőrendű logika közötti különbség	11
2.1.4. Eldönthetőség	11
2.1.5. Klasszikus és konstruktív logika	12
3. Modell fogalom	15
3.1. Kategória	15
3.2. Termék	16
3.2.1. De Bruijn-indexek	17
3.3. Formulák	19
3.4. Bizonyítások	20
3.5. Reláció- és függvénszimbólumok	20
3.6. Logikai összekötők	21
3.6.1. Implikáció ($\supset$)	21
3.6.2. Konjunkció ($\wedge$)	21

3.6.3.	Diszjunkció ($\vee$)	22
3.6.4.	Konstans igaz ($\top$)	22
3.6.5.	Konstans hamis ($\perp$)	23
3.6.6.	Kvantorok	23
4.	Szintaxis	25
5.	Modellek	31
5.1.	Bool modell	31
5.2.	Tarski modell	32
5.3.	Family modell	32
5.4.	Modellek összehasonlítása	33
5.5.	Összegzés	33
6.	Összegzés	34

1. fejezet

Bevezetés

1.1. A dolgozat célja

A dolgozat célja az elsőrendű logika formalizálása Agdában, mint mélyen beágyazott nyelv. A mély beágyazás biztosítja, hogy teljes kontrollunk legyen a logika reprezentációja felett. Többféle szemantikát meg tudunk így adni, míg a sekély beágyazás rögzít egy szemantikát, mivel közvetlenül a metanyelv konstrukcióit használja.

Ebben a dolgozatban az elsőrendű logika egy általánosított algebrai elméletként (GAT) lett megvalósítva, így a modell fogalmat többszortú módon adjuk, amelyek indexelve vannak egymás felett. A szortok a következők lesznek: **Tm**, **For**, **Pf**. Így reperezántálva a kifejezéseket, formulákat, és bizonyításokat. A változók miatt pedig a **Con** és **Sub** szortokat vezetjük be, ábrázolva a környezeteket és a közöttük való átjárást.

A fő kontribúció abban rejlik, hogy az iniciális algebrát kvóciensek nélkül (induktív adattípusokkal) adjuk meg. Erre egy jó példa lehet az A feletti monoid, ami egy egyenlőséges algebrai elmélet, de meg tudjuk adni a szintaxisát egyenlőségek nélkül.

```
record Monoid {A : Set} : Set1 where
  field
    C : Set
     $\_ \bullet \_ : C \rightarrow C \rightarrow C$ 
    assoc : {x y z : C}  $\rightarrow (x \bullet y) \bullet z \equiv x \bullet (y \bullet z)$ 
    e : C
```

$$\begin{aligned} \text{idl} &: \{x : \mathbf{C}\} \rightarrow \mathbf{e} \bullet x \equiv x \\ \text{idr} &: \{x : \mathbf{C}\} \rightarrow x \bullet \mathbf{e} \equiv x \\ \eta &: A \rightarrow \mathbf{C} \end{aligned}$$

data List (A : Set) : Set **where**

[] : List A

_ _ : A → List A → List A

++ : ∀ {A} → List A → List A → List A

[] ++ ys = ys

(x xs) ++ ys = x (xs ++ ys)

assoc : ∀ {A} {x y z : List A} → (x ++ y) ++ z ≡ x ++ (y ++ z)

assoc {A} {[]} {y} {z} = **refl**

assoc {A} {x x₁} {y} {z} = **cong** (x **_**) (**assoc** {A} {x₁})

idr : ∀ {A} {x : List A} → x ++ [] ≡ x

idr {A} {[]} = **refl**

idr {A} {x x₁} = **cong** (x **_**) (**idr** {A} {x₁})

l : {A : Set} → Monoid {A}

l {A} = **record**

{ **C** = List A

; **_•_** = **_++_**

; **assoc** = λ {x} {y} {z} → **assoc** {A} {x} {y} {z}

; **e** = []

; **idl** = **refl**

; **idr** = **idr**

; **η** = **_** []

}

A monoid szintaxisaként itt a listát használjuk, ugyanis a listák és a konkatenáció természetes módon kielégítik a monoid egyenlőségeit (asszociativitás, identitáselem), így a konstrukció definícionálisan garantálja ezeket a tulajdonságokat.

A következő konstrukció ellenpéldaként szolgál, nevezhetjük rendezetlen párnak:

```

record UP {A : Set} : Set1 where
  field
    C : Set
    _,_ : A → A → C
    eq : {x y : A} → x , y ≡ y , x

```

Ahhoz, hogy megadjuk a rendezetlen pár szintaxisát, egy kiegészítő egyenletre van szükség, amely biztosítja a kommutativitást $(a, b) \equiv (b, a)$.

Míg bizonyos algebrai struktúrák iniciális modelljei definiálhatók egyenlőségek megadása nélkül, tisztán olyan módon, hogy az egyenlőségek definícionálisan teljesüljenek, másokhoz további egyenletek szükségesek a kívánt tulajdonságok kikényszerítésére.

Egy algebrai elméletet szortok, operátorok és egyenletek határoznak meg. Egy elmélet modelljeit vagy algebrait algebrai struktúrának is szokás nevezni, ez minden szorthoz egy halmazt, az operátorokhoz függvényeket, az egyenletekhez pedig egyenlőséget rendel.

Például a félcsoport egy olyan algebrai struktúra, amely egy halmazból és a rajta értelmezett asszociatív bináris műveletből áll. Ebben az esetben az asszociativitás lesz az az egyenlőség, amelyet a félcsoportnak teljesítenie kell.

```

record Semigroup : Set1 where
  field
    C : Set
    _•_ : C → C → C
    associativity : (x y z : C) → (x • y) • z ≡ x • (y • z)

```

Ezek alapján a $\mathbb{N}$ és a $+$ (összeadás), vagy a $\mathbb{B}$ (Bool) és az $\wedge$ (és) félcsoportot alkotnak. Ez nem mondható el viszont a $\mathbb{N}$ és a $-$ (kivonás) párosításról, ugyanis az utóbbi műveletre nem igaz, hogy asszociatív.

Az előbb említett félcsoportokat egy-egy modellként is reprezentálhatjuk a következő módon:

```

N+ : Semigroup
N+ = record
  { C = N
    ; _•_ = _+_

```

```
; associativity = +-assoc  
}
```

A szintaxist definiálhatjuk egy olyan modellként, amelyből minden más modellbe megy egy függvény megőrizve az operátorokat. A félcsoport szintaxisát a 0 elemű halmazzal tudjuk megadni. A 0 elemű halmazon meg tudunk adni mintaillesztéssel tetszőleges más halmazba függvényt, amely triviálisan megőrzi az összes operátort.

```
data C : Set where  
  
I : Semigroup  
I = record  
{ C = C  
; _•_ =  $\lambda x y \rightarrow x$   
; associativity =  $\lambda x y z \rightarrow \text{refl}$   
}
```

1.2. Elsőrendű logika Agdában

Az Agda egy függően típusozott funkcionális nyelv, amelyet bizonyításokra is tudunk használni, ezzel megmutatva a logika kapcsolatát a típuselmélettel. Továbbá egy teljes nyelv is, ami azt jelenti, hogy minden program terminál és mintaillesztés során is minden esetet kötelező lekezelni.

Ahogy már korábban szó volt róla, az elsőrendű logikát mint nyelvet sekély módon is be lehet ágyazni az Agda rendszerébe, mivel ehhez minden eszköz adott a meglévő típusok és műveletek által.

Az elsőrendű logika sekély beágyazása Agdában meglehetősen természetesnek is hathat:

1.2.1. Logikai állítások, mint típusok

Az elsőrendű logikában egy formula, mint $P(x)$, vagy igaz, vagy hamis. Agdában kezelhetjük az állításokat típusokként, így a bizonyítása a $P(x)$ formulának megfeleltethető az állítás szort egy elemének.

1.2.2. Az univerzális kvantor, mint egy függő függvény típus

Az univerzális kvantor $\forall x.P(x)$ egyszerűen reprezentálható egy függő függvény típussal:

```

 $\forall\_ : \{A : \text{Set}\} \rightarrow (A \rightarrow \text{Prop}) \rightarrow \text{Prop}$ 
 $\forall P = \forall x \rightarrow P\ x$ 

```

1.2.3. Az egzisztenciális kvantor ($\exists$), mint a függő pár típus (Σ)

Az egzisztenciális kvantor $\exists x.P(x)$ megfeleltethető a függő pár típusnak:

```

data  $\exists$  (A : Set) (B : A  $\rightarrow$  Prop) : Prop where
   $\_ , \_ : (a : A) \rightarrow B\ a \rightarrow \exists\ A\ B$ 

```

1.2.4. Logikai összekötők, mint meglévő típuskonstruktorok

- Konjunkció ($\wedge$) - szorzat típus (Σ -types):

```

record  $\_ \Sigma \_$  (A B : Set) : Set where
  constructor  $\langle \_, \_ \rangle$ 
  field
    fst : A
    snd : B

```

- Disjunkció ($\vee$) - összeg típus:

```

data  $\_ \uplus \_$  (A B : Set) : Set where
  inl : A  $\rightarrow$  A  $\uplus$  B
  inr : B  $\rightarrow$  A  $\uplus$  B

```

- Implikáció ($\rightarrow$) - függvény típus:

```

 $\_ \rightarrow \_ : \text{Set} \rightarrow \text{Set} \rightarrow \text{Set}$ 
A  $\rightarrow$  B = A  $\rightarrow$  B -- Az Agda függvény típusa

```

- **Negáció** ($\neg A$) - $A \rightarrow \perp$ (ahol $\perp$ az üres típus):

$$\neg_ : \text{Set} \rightarrow \text{Set}$$

$$\neg A = A \rightarrow \perp$$

1.2.5. Egyenlőség és elsőrendű termék

A beépített egyenlőség Agdában ($_ \equiv _$), amelyet közvetlenül használhatunk:

postulate

$$_ \equiv _ : \{A : \text{Set}\} \rightarrow A \rightarrow A \rightarrow \text{Set}$$

Mindezekkel a funkciókkal a sekély beágyazás egyszerűen megvalósítható. Az elsőrendű logikát nem szükséges szintaktikus fákként kódolni, hanem egyszerűen a beépített Agda típusokat és függvényeket használhatjuk.

1.2.6. Példa bizonyítás

Például, ha be akarnánk bizonyítani a következőt:

$$\forall x. (P(x) \rightarrow Q(x)) \rightarrow (\forall x. P(x)) \rightarrow (\forall x. Q(x))$$

Egyszerűen csak egy függvény applikációval adjuk meg:

$$\text{lemma} : \{A : \text{Set}\} \{x : A\} \{P Q : A \rightarrow \text{Set}\} \rightarrow$$

$$(\forall x \rightarrow P x \rightarrow Q x) \rightarrow (\forall x \rightarrow P x) \rightarrow \forall x \rightarrow Q x$$

$$\text{lemma } f \ g \ x = f \ x \ (g \ x)$$

2. fejezet

Logika

2.1. Bevezetés a logikába

A logika a helyes következtetés és érvelés tudománya. Matematikai szempontból a logika egy olyan formális nyelv vagy rendszer, amely lehetővé teszi az állítások precíz leírását, valamint ezek igazságértékének és egymáshoz való viszonyának vizsgálatát. A formális logika célja, hogy matematikailag megragadható módon írjon le következtetéseket, amelyek objektív, jól meghatározott szabályokon alapulnak.

A különböző típusú logikák eltérő kifejezőerővel rendelkeznek. A legegyszerűbb formális logikai rendszer a nulladrendű logika, más néven propozíciós logika, amelyet a predikátumlogika vagy elsőrendű logika követ, amely már képes az egyedekről és azok tulajdonságairól is beszélni.

2.1.1. Propozíciós (nulladrendű) logika

A propozíciós logika olyan állításokkal dolgozik, amelyek teljes egészükben igazak vagy hamisak, és nem bonthatók tovább kisebb részekre anélkül, hogy elveszítenék jelentésüket. Ezeket az állításokat propozícióknak nevezzük, és általában kisbetűkkel jelöljük, például p , q , r .

A propozíciókhoz logikai műveletek tartoznak, mint például:

- Negáció ($\neg p$) – "nem p "
- Konjunkció ($p \wedge q$) – " p és q "
- Diszjunkció ($p \vee q$) – " p vagy q "

- Implikáció ($p \rightarrow q$) – "ha p , akkor q "
- Ekvivalencia ($p \leftrightarrow q$) – " p akkor és csak akkor, ha q "

A propozíciós logika célja annak eldöntése, hogy bizonyos formulák egy adott logikai rendszerben igazak-e, illetve hogy egyes állítások logikailag következnek-e másokból.

2.1.2. Predikátumlogika (elsőrendű logika)

Az elsőrendű logika tovább lép a propozíciós logika korlátain. Itt már nem csupán állításokat vizsgálunk, hanem azok belső szerkezetét is feltárjuk. A predikátumlogika képes kifejezni olyan állításokat, mint:

- "Minden ember halandó"
- "Van olyan szám, amely páros"

Ez úgy éri el, hogy egyedekre (individuumokra), tulajdonságokra (predikátumokra) és kapcsolatokra épít. A rendszer alapelemei:

- Változók (pl. ' x ', ' y ', ' z ')
- Konstansok (pl. ' a ', ' b ', ' 0 ', ' 1 ')
- Függvényszimbólumok (pl. ' $f(x)$ ')
- Predikátumszimbólumok (pl. ' $\text{Ember}(x)$ ', ' $\text{Szeret}(x, y)$ ')
- Kvantorok:
 - Univerzális kvantor ($\forall x$) – "minden x -re"
 - Egzisztenciális kvantor ($\exists x$) – "létezik olyan x "

Például a mondat:

Minden ember halandó

formalizálva így néz ki:

$$\forall x. \text{Ember}(x) \rightarrow \text{Halandó}(x)$$

2.1.3. A propozíciós és az elsőrendű logika közötti különbség

A két logikai rendszer között a fő különbség az elemzési mélységben és kifejezőerőben rejlik:

Tulajdonság	Propozíciós logika	Elsőrendű logika
Alapegység	Egész állítások (pl. "esik az eső")	Egyedek, tulajdonságok és relációk
Belső szerkezet	Nem vizsgálja	Vizsgálja és kezeli
Kvantorok	Nincsenek	Vannak ($\forall$, $\exists$)
Kifejezőerő	Korlátozott	Sokkal gazdagabb
Példa	$p \rightarrow q$	$\forall x. \text{Ember}(x) \rightarrow \text{Halandó}(x)$

2.1. táblázat. A propozíciós és az elsőrendű logika közötti különbségek.

A propozíciós logika tehát alkalmas egyszerű állítások kezelésére, míg az elsőrendű logika képes összetettebb szerkezeteket és általános állításokat is kezelni. Ennek következtében az elsőrendű logika az automatikus bizonyítás, formális verifikáció és matematikai elméletek formalizálásának is egyik alapja. Például Agdában is ezen alapul a legtöbb formális modell.

2.1.4. Eldönthetőség

A logikai rendszerek egyik központi kérdése az eldönthetőség. Egy logikai rendszer eldönthető, ha létezik olyan algoritmus, amely minden lehetséges bemenetre, véges időn belül eldönti, hogy az adott formula érvényes vagy nem, vagy hogy egy adott formula levezethető-e a rendszer axiómáiból és szabályaiból.

Ez különösen fontos a formális verifikációban és az automatikus tételbizonyításban, ahol azt szeretnénk, hogy egy gép automatikusan meg tudja mondani, hogy egy adott logikai állítás igaz-e vagy sem.

Eldönthetőség a propozíciós logikában

A propozíciós logika eldönthető: létezik algoritmus, amely képes eldönteni bármely propozíciós formula esetén, hogy az logikailag érvényes-e. Az egyik ilyen algoritmus például az igazságtábla-módszer, amely minden lehetséges értékkombinációt kipróbál.

Vegyünk például az alábbi formulát:

$$(p \wedge (p \rightarrow q)) \rightarrow q$$

Ez egy tautológia, azaz mindig igaz, és ezt géppel is eldönthetjük véges időn belül.

Eldönthetőség az elsőrendű logikában

Az elsőrendű logika általánosságban nem eldönthető. Ez azt jelenti, hogy nem létezik olyan algoritmus, amely minden elsőrendű formula esetén képes lenne megmondani véges időn belül, hogy az levezethető-e vagy sem.

Ez a híres Church–Turing-tétel egyik következménye, amely kimondja, hogy az elsőrendű logika bizonyíthatósága rekurzívan enumerálható, de nem rekurzívan eldönthető.

Vannak azonban esetek, ahol eldönthetőség biztosítható, ha valamilyen formában korlátokat vezetünk be, például formulakészletek szűkítése, vagy véges struktúrákra korlátozott értelmezések esetén.

Az eldönthetőség hiánya azt jelenti, hogy az elsőrendű logikában nem mindig lehet automatizálni a bizonyítást. Egy automatikus tételbizonyító rendszer, például egy Agdához hasonló típuselmélet-alapú nyelv, nem garantálhatja, hogy minden helyes állítást képes lesz levezetni. Az ilyen rendszerek gyakran fél-automaták: bizonyos esetekben képesek segíteni vagy teljesen elvégezni a bizonyítást, de általános megoldás nem létezik.

2.1.5. Klasszikus és konstruktív logika

A klasszikus logika a legelterjedtebb, amelyet a legtöbb hagyományos matematikai elmélet is használ. Ezzel szemben a konstruktív logika – más néven intuicionista logika – eltér bizonyos alapvető logikai szabályok elfogadásában, különösen azokra az esetekre vonatkozóan, amikor valaminek a létezését vagy valami igazságát állítjuk anélkül, hogy konkrét példát tudnánk adni vagy bizonyítékot tudnánk konstruálni.

A klasszikus logika főbb jellemzői

A klasszikus logika elfogadja az alábbi törvényeket:

- Kizárt harmadik elve: $P \vee \neg P$ minden P formula esetén igaz.
- Kettős negáció elve: $\neg\neg P \rightarrow P$ minden P formula esetén igaz.

- Nem-konstruktív létezés bizonyítása: elég kimutatni, hogy a létezés tagadása ellentmondáshoz vezet.

Ezek az elvek lehetővé teszik sok klasszikus matematikai bizonyításfajta alkalmazását, például:

- Indirekt bizonyítás (reductio ad absurdum)
- Nem konstruktív létezés bizonyítása (pl. "létezik irracionális szám 'a', amire a^a racionális" – de nem mondjuk meg, melyik az)

A konstruktív logika főbb jellemzői

A konstruktív logika nem fogadja el automatikusan a kizárt harmadik elvét, sem a kettős negáció elvét. Ehelyett csak akkor állíthatjuk, hogy egy állítás igaz:

- ha konkrét bizonyítékot tudunk adni rá (pl. konstrukciót vagy algoritmust),
- vagy ha például $\exists x. P(x)$ állítást vizsgáljuk, akkor meg kell mutatni konkrétan az 'x'-et, amire $P(x)$ igaz.

Ez a logika különösen jól illeszkedik a kategóriaelmélethez, típuselmélethez és a formális bizonyítási rendszerekhez. Ezekben a rendszerekben egy állítás bizonyítása gyakran azt jelenti, hogy megadunk egy konkrét értéket vagy programot, amely megfelel az állításnak.

Állítás	Klasszikus logika szerint	Konstruktív logika szerint
$P \vee \neg P$	Mindig igaz	Nem mindig igaz (bizonyíték kell valamelyik ágra)
$\neg\neg P \rightarrow P$	Igaz	Nem elfogadott általánosan
$\exists x.P(x)$	Elég indirekt bizonyítás	Csak konkrét x bemutatása mellett igaz

2.2. táblázat. Különbségek a klasszikus és konstruktív logika között.

Kapcsolat az Agdával

Az Agda és más hasonló rendszerek (pl. Coq, Lean) konstruktív logikát alkalmaznak: amikor egy állítást bebizonyítunk Agdában, valójában egy értéket konstruálunk, amely az adott állítás típusának felel meg. Ennek alapja a Curry–Howard megfelelés, amely szerint:

- Állítások $\leftrightarrow$ Típusok

- Bizonyítások $\leftrightarrow$ Programok (értékek)

Ez a szemlélet a konstruktív logikához illeszkedik, hiszen minden állításnak csak akkor van "értelme", ha tudunk adni rá konkrét példát, vagy meg tudjuk konstruálni a bizonyítását.

3. fejezet

Modell fogalom

3.1. Kategória

A modell fogalom első elemeként megadunk egy kontextusok és behelyettesítések által alkotott kategóriát. A kategória objektumait a kontextusok reprezentálják és a **Con** szort határozza meg. A morfizmusai pedig a behelyettesítések lesznek, amelyeket a **Sub** szort ad meg.

$$\begin{aligned}\text{Con} &: \text{Set } i \\ \text{Sub} &: \text{Con} \rightarrow \text{Con} \rightarrow \text{Set } (k \sqcup l)\end{aligned}$$

A kategóriákban a morfizmusoknak kell rendelkezniük egy kompozíciós művelettel ($_ \circ _$) is, amely két morfizmus összekapcsolását végzi el, ezzel létrehozva egy új morfizmust.

$$\begin{aligned}_ \circ _ &: \forall \{\Gamma \Delta \Theta\} \rightarrow \text{Sub } \Delta \Gamma \rightarrow \text{Sub } \Theta \Delta \rightarrow \text{Sub } \Theta \Gamma \\ \text{ass} &: \forall \{\Gamma \Delta \Theta \Xi\} \{\gamma : \text{Sub } \Delta \Gamma\} \{\delta : \text{Sub } \Theta \Delta\} \{\theta : \text{Sub } \Xi \Theta\} \\ &\rightarrow (\gamma \circ \delta) \circ \theta \equiv \gamma \circ (\delta \circ \theta)\end{aligned}$$

Tehát, ha van egy morfizmus a Δ és Γ kontextusok között, valamint egy másik morfizmus a Θ és Δ között, akkor a kompozíciójuk egy új morfizmust ad a Θ és Γ kontextusok között. A kompozíciónak továbbá asszociatívnak kell lennie, amit az **ass** fejez ki.

A kategóriában minden objektumhoz léteznie kell egy olyan morfizmusnak, amely az objektumot önmagába képezi, ez lesz az identitás. Az identitás morfizmusnak a kompozícióval való alkalmazása nem változtathatja meg a kompozícióban szereplő másik morfizmust. Ezt az **idl** és **idr** egyenlőség biztosítja.

$$\begin{aligned}
 \text{id} &: \forall\{\Gamma\} \rightarrow \text{Sub } \Gamma \ \Gamma \\
 \text{idl} &: \forall\{\Gamma \ \Delta\}\{\gamma : \text{Sub } \Delta \ \Gamma\} \rightarrow \text{id} \circ \gamma \equiv \gamma \\
 \text{idr} &: \forall\{\Gamma \ \Delta\}\{\gamma : \text{Sub } \Delta \ \Gamma\} \rightarrow \gamma \circ \text{id} \equiv \gamma
 \end{aligned}$$

A terminális objektum egy olyan objektum, amelyhez minden más objektumból létezik pontosan egy morfizmus. A mi esetünkben ez a terminális objektum a $\diamond$, a hozzá tartozó morfizmus pedig az ε . A morfizmus egyedülállóságát a $\diamond\eta$ fogalmazza meg.

$$\begin{aligned}
 \diamond &: \text{Con} \\
 \varepsilon &: \forall\{\Gamma\} \rightarrow \text{Sub } \Gamma \ \diamond \\
 \diamond\eta &: \forall\{\Gamma\}\{\sigma : \text{Sub } \Gamma \ \diamond\} \rightarrow \sigma \equiv \varepsilon
 \end{aligned}$$

3.2. Termek

A termeket (Tm) egy funktorként adjuk meg. Úgy is mondhatjuk, hogy egy kontextustól függenek, mivel lehetnek benne termváltozók, amikbe be tudunk majd helyettesíteni, így értéket adva a szabad változóknak. Ehhez szükségünk lesz termek-be való behelyettesítésre, másnéven egy morfizmusok feletti akcióra. Egy funktornak teljesítenie kell két feltételt is: a kompozícióval való kompatibilitást és az identitással való kompatibilitást. Ezeket az $[\circ]^t$ és $[\text{id}]^t$ egyenlőségek biztosítják.

$$\begin{aligned}
 \text{Tm} &: \text{Con} \rightarrow \text{Set } k \\
 []^t &: \forall\{\Gamma \ \Delta\} \rightarrow \text{Tm } \Gamma \rightarrow \text{Sub } \Delta \ \Gamma \rightarrow \text{Tm } \Delta \\
 [\circ]^t &: \forall\{\Gamma \ \Delta \ \theta\}\{t : \text{Tm } \Gamma\}\{\gamma : \text{Sub } \Delta \ \Gamma\}\{\delta : \text{Sub } \theta \ \Delta\} \\
 &\rightarrow t \ [\gamma \circ \delta]^t \equiv t \ [\gamma]^t \ [\delta]^t \\
 [\text{id}]^t &: \forall\{\Gamma\}\{t : \text{Tm } \Gamma\} \rightarrow t \ [\text{id}]^t \equiv t
 \end{aligned}$$

A környezetek termekkel és termváltozókkal való kiegészítését is megadjuk. A környezetbe betenni egy termváltozót ($_ \triangleright_t$) hasonlítható egy olyan Descartes szorzathoz, amelynek az egyik oldalán az egyelemű halmaz található: $_ \times \top$. A behelyettesítéseket is kiegészíthetjük termekkel ($_ \triangleright_t _$), amelyeket ismételten a Descartes szorzattal lehetne azonosítani. A képzett pároknak tehát meg tudjuk adni az első ($\mathbf{p}_t$) és a második ($\mathbf{q}_t$) projekcióját.

$$\begin{aligned}
 _ \triangleright_t &: \text{Con} \rightarrow \text{Con} \\
 _ \triangleright_t _ &: \forall\{\Gamma \ \Delta\} \rightarrow \text{Sub } \Delta \ \Gamma \rightarrow \text{Tm } \Delta \rightarrow \text{Sub } \Delta \ (\Gamma \triangleright_t)
 \end{aligned}$$

$$\begin{aligned} p_t &: \forall \{\Gamma\} \rightarrow \text{Sub}(\Gamma \triangleright_t) \Gamma \\ q_t &: \forall \{\Gamma\} \rightarrow \text{Tm}(\Gamma \triangleright_t) \end{aligned}$$

Fontos megadjuk a redukciós szabályokat is (β, η) , hogy biztosítsuk az azonos kifejezések közötti egyenlőséget.

$$((p \circ _) , (q[_])) : \text{Sub} \Delta (\Gamma \triangleright_t) \cong \text{Sub} \Delta \Gamma \times \text{Tm} \Delta : _, t _$$

- β : jobbról balra, majd jobbra = mintha nem történt volna semmi
- η : balról jobbra, majd balra = mintha nem történt volna semmi

$$\begin{aligned} \triangleright_t \beta_1 &: \forall \{\Gamma \Delta\} \{t : \text{Tm} \Delta\} \{\gamma : \text{Sub} \Delta \Gamma\} \rightarrow p_t \circ (\gamma _, t) \equiv \gamma \\ \triangleright_t \beta_2 &: \forall \{\Gamma \Delta\} \{t : \text{Tm} \Delta\} \{\gamma : \text{Sub} \Delta \Gamma\} \rightarrow q_t [\gamma _, t]^t \equiv t \\ \triangleright_t \eta &: \forall \{\Gamma \Delta\} \rightarrow \{\gamma t : \text{Sub} \Delta (\Gamma \triangleright_t)\} \rightarrow \gamma t \equiv (p_t \circ \gamma t _, q_t [\gamma t]^t) \end{aligned}$$

3.2.1. De Bruijn-indexek

A De Bruijn-indexek a változók lambda-kalkulus vagy logikai kifejezésekben való ábrázolásának egy módja. Fő előnye, hogy nem használ változóneveket. Ahelyett, hogy egy változóra névvel hivatkoznánk, minden változót egy számmal reprezentálunk, amely megadja, hogy hány kötőelemet (például kvantorokat) kell felfelé haladnunk a kifejezésfában, hogy megtaláljuk a változó értékét. A De Bruijn-indexek használata megkönnyíti a kifejezések manipulálását és egyszerűsíti a bizonyításokat, mivel nem fordul elő névütközés vagy változó-újradefiniálás.

Egy egyszerű példa:

A $\lambda x. \lambda y. x$ kifejezés De Bruijn-indexekkel: $\lambda. \lambda. 1$.

Az 1 itt a két szinttel feljebb kötött változóra hivatkozik (0-tól indexelve), azaz az x -re.

Az indexelés a következőképp működik:

- A legbelső kötött változó a 0 indexet kapja.
- Az azt követő az 1 indexet.
- Aztán 2 és így tovább.

Tehát:

- A $\lambda x. x$ kifejezés $\lambda. 0$ lesz.
- A $\lambda x. \lambda y. x$ kifejezés $\lambda. \lambda. 1$ lesz.
- A $\lambda x. \lambda y. y$ kifejezés $\lambda. \lambda. 0$ lesz.

Miért "jobbak" a De Bruijn-indexek?

Különösen az olyan formális rendszerekben, mint jelen esetben az elsőrendű logikai implementáció, a De Bruijn-indexek segítenek a következőkben:

- A változónevek ütközésének elkerülése

Egy helyettesítés végrehajtásakor az átnevezés (renaming, alfa konverzió) szükségtelenné válik. Nem kell többé aggódni a változók véletlen elfogása (variable capture) miatt.

- Helyettesítések egyszerűbb végrehajtása

A helyettesítés következetessé válik. Nincs szükség a változók nevének nyomon követésére, csak az indexeket kell megfelelően eltolni a helyettesítés során.

- Egyszerűbb egyenlőségellenőrzés

Két kifejezés az átnevezésig egyenlő, ha De Bruijn-ábrázolásaik megegyeznek. Mondhatni ingyen kapjuk az alfa-ekvivalenciát.

- Jobban illik a mechanikus bizonyítási rendszerekhez Az olyan rendszerek, mint az Agda, a Coq vagy a logikát belsőleg implementáló bizonyítási asszisztensek előnyben részesülnek a De Bruijn-indexek használatával kapcsolatban, mert:

- Függő típusos nyelvekben könnyebb őket megvalósítani
- A mintaillesztés is kevésbé érzékeny a hibákra
- Passzol a strukturális rekurziós elvekhez az olyan totális nyelvekben, mint az Agda.

Az univerzális kvantor szignatúrája tipikusan a következőképpen nézne ki:

$\forall$

— : Formula $\rightarrow$ Formula

De szemantikusan, a De Bruijn-indexelt környezetben, a $\forall$ köt egy változót a formulában, ami azt eredményezi, hogy egyel kevesebb szabad változó lesz benne, ami a szignatúrában is látszik:

$$\forall : (A : \text{For } \Gamma(\triangleright_t)) \rightarrow \text{For } \Gamma$$

Összességében esetünkben nagyban megkönnyíti a munkát a

- $\forall$ és $\exists$ implementálásánál és használatánál
- behelyettesítésekkel kapcsolatos műveleteknél
- formális bizonyításoknál

Jelen implementációban a De Bruijn-indexek megléte a következőképpen valósul meg:

- A $_ \triangleright_t$ művelet azáltal ad hozzá egy új változót a környezethez, hogy az indexeket eltolja egyel.
- A $_ \triangleright_t _$ megfeltethető azzal, hogy az új változóhoz konstruálunk egy behelyettesítést.
- A p_t és q_t pedig a projekciók, amelyek segítenek lebontani a behelyettesítést.
 - p_t eldobja a legújabb változót
 - q_t a változó a 0. indexen

Ugyanez a megvalósítás látható a későbbiekben a bizonyítások esetében is.

3.3. Formulák

A formulákra is tekinthetünk funktorként, így hasonlóan a termekhez a következő módon adjuk meg:

$$\begin{aligned} \text{For} &: \text{Con} \rightarrow \text{Set } j \\ _ _ _ &^f : \forall \{ \Gamma \Delta \} \rightarrow \text{For } \Gamma \rightarrow \text{Sub } \Delta \Gamma \rightarrow \text{For } \Delta \\ _ \circ _ &^f : \forall \{ \Gamma \Delta \theta \} \{ A : \text{For } \Gamma \} \{ \gamma : \text{Sub } \Delta \Gamma \} \{ \delta : \text{Sub } \theta \Delta \} \\ &\rightarrow A _ [\gamma \circ \delta]^f \equiv A _ [\gamma]^f [\delta]^f \\ \text{[id]} &^f : \forall \{ \Gamma \} \{ A : \text{For } \Gamma \} \rightarrow A _ \text{[id]}^f \equiv A \end{aligned}$$

3.4. Bizonyítások

A bizonyításokat már egy kicsivel bonyolultabban adjuk meg, ugyanis egy **For** feletti dependáns funktronak is nevezhetnénk. Ha vesszük például a $\Gamma \vdash A$ szokásos logikai (bizonyításelméleti) jelölést, akkor a $\mathbf{p} : \mathbf{Pf} \Gamma A$ azt jelenti, hogy $\mathbf{p}$ bizonyítja az A állítást, ahol a szabad változók Γ -ban vannak.

$$\begin{aligned} \mathbf{Pf} &: (\Gamma : \mathbf{Con}) \rightarrow \mathbf{For} \Gamma \rightarrow \mathbf{Prop} \, l \\ _ \llbracket _ \rrbracket^p &: \forall \{\Gamma \Delta A\} \rightarrow \mathbf{Pf} \Gamma A \rightarrow (\gamma : \mathbf{Sub} \Delta \Gamma) \rightarrow \mathbf{Pf} \Delta (A \llbracket \gamma \rrbracket^f) \end{aligned}$$

A bizonyítás-változókat a termváltozók esetében is használt operációkhoz hasonlóan valósítjuk meg:

$$\begin{aligned} _ \triangleright_p _ &: (\Gamma : \mathbf{Con}) \rightarrow \mathbf{For} \Gamma \rightarrow \mathbf{Con} \\ _ \triangleright_p _ &: \forall \{\Delta \Gamma A\} \rightarrow (\gamma : \mathbf{Sub} \Delta \Gamma) \rightarrow \mathbf{Pf} \Delta (A \llbracket \gamma \rrbracket^f) \rightarrow \mathbf{Sub} \Delta (\Gamma \triangleright_p A) \\ \mathbf{p}_p &: \forall \{\Gamma A\} \rightarrow \mathbf{Sub} (\Gamma \triangleright_p A) \Gamma \\ \mathbf{q}_p &: \forall \{\Gamma A\} \rightarrow \mathbf{Pf} (\Gamma \triangleright_p A) ((A \llbracket \mathbf{p}_p \rrbracket^f)) \\ \triangleright_p \beta_1 &: \forall \{\Gamma \Delta A\} \{\gamma : \mathbf{Sub} \Delta \Gamma\} \{a : \mathbf{Pf} \Delta (A \llbracket \gamma \rrbracket^f)\} \rightarrow \mathbf{p}_p \circ (\gamma \triangleright_p a) \equiv \gamma \\ \triangleright_p \eta &: \forall \{\Gamma \Delta A\} \{\gamma a : \mathbf{Sub} \Delta (\Gamma \triangleright_p A)\} \\ &\rightarrow \gamma a \equiv (\mathbf{p}_p \circ \gamma a \triangleright_p \mathbf{substP} (\mathbf{Pf} \Delta) ([\circ]^f)^{-1}) (\mathbf{q}_p \llbracket \gamma a \rrbracket^p) \\ ((\mathbf{p}_p \circ _), (\mathbf{q}_p \llbracket _ \rrbracket^p)) &: \mathbf{Sub} \Delta (\Gamma \triangleright_p A) \cong (\gamma : \mathbf{Sub} \Delta \Gamma) \times \mathbf{Pf} \Delta (A \llbracket \gamma \rrbracket^f) : _ \triangleright_p _ \end{aligned}$$

3.5. Reláció- és függvényszimbólumok

A modell a függvények és a relációk arításával van felparaméterezve, ezt felhasználva határozzuk meg, hogy egy reláció vagy függvény hány termet vár majd paraméterként.

$$\begin{aligned} \mathbf{Rel} &: \forall \{\Gamma\} \{n : \mathbb{N}\} \rightarrow \mathbf{relar} \, n \rightarrow \mathbf{Tm} \Gamma \wedge n \rightarrow \mathbf{For} \Gamma \\ \mathbf{Rel} \llbracket _ \rrbracket &: \forall \{\Gamma \, n\} \{ar : \mathbf{relar} \, n\} \{ts : \mathbf{Tm} \Gamma \wedge n\} \{\Delta\} \{\gamma : \mathbf{Sub} \Delta \Gamma\} \\ &\rightarrow \mathbf{Rel} \, ar \, ts \llbracket \gamma \rrbracket^f \equiv \mathbf{Rel} \, ar \, (\mathbf{map} _ \llbracket \gamma \rrbracket^t \, ts) \\ \mathbf{fun} &: \forall \{\Gamma\} \{n : \mathbb{N}\} \rightarrow \mathbf{funar} \, n \rightarrow \mathbf{Tm} \Gamma \wedge n \rightarrow \mathbf{Tm} \Gamma \\ \mathbf{fun} \llbracket _ \rrbracket &: \forall \{\Gamma \, n\} \{ar : \mathbf{funar} \, n\} \{ts : \mathbf{Tm} \Gamma \wedge n\} \{\Delta\} \{\gamma : \mathbf{Sub} \Delta \Gamma\} \\ &\rightarrow \mathbf{fun} \, ar \, ts \llbracket \gamma \rrbracket^t \equiv \mathbf{fun} \, ar \, (\mathbf{map} _ \llbracket \gamma \rrbracket^t \, ts) \end{aligned}$$

3.6. Logikai összekötők

3.6.1. Implikáció ($\supset$)

Az $A \supset B$ jelentése: "ha A igaz, akkor B is igaz".

$$_ \supset _ : \forall \{\Gamma\} \rightarrow \text{For } \Gamma \rightarrow \text{For } \Gamma \rightarrow \text{For } \Gamma$$

$$\supset [] : \forall \{\Gamma \ A \ B \ \Delta\} \{\gamma : \text{Sub } \Delta \ \Gamma\} \rightarrow (A \supset B) \ [\gamma]^f \equiv A \ [\gamma]^f \supset B \ [\gamma]^f$$

- Az implikáció bevezető szabálya ($\supset \text{in}$): Ha tudjuk bizonyítani, hogy B igaz egy olyan kontextusban, ahol A feltételezett, akkor van egy bizonyításunk $A \supset B$ -re is.

$$\supset \text{in} : \forall \{\Gamma \ A \ B\} \rightarrow \text{Pf } (\Gamma \triangleright_p A) (B \ [\text{p}_p]^f) \rightarrow \text{Pf } \Gamma ((A \supset B))$$

- Az implikáció kivezető szabálya ($\supset \text{out}$): Ha van egy bizonyításunk $A \supset B$ -re, valamint A-ra is, akkor van egy bizonyításunk B-re is.

$$\supset \text{out} : \forall \{\Gamma \ A \ B\} \rightarrow \text{Pf } \Gamma (A \supset B) \rightarrow \text{Pf } \Gamma A \rightarrow \text{Pf } \Gamma B$$

3.6.2. Konjunkció ($\wedge$)

Az $A \wedge B$ azt jelenti, hogy mind A, mind B igaz.

$$_ \wedge _ : \forall \{\Gamma\} \rightarrow \text{For } \Gamma \rightarrow \text{For } \Gamma \rightarrow \text{For } \Gamma$$

$$\wedge [] : \forall \{\Gamma \ A \ B \ \Delta\} \{\gamma : \text{Sub } \Delta \ \Gamma\} \rightarrow (A \wedge B) \ [\gamma]^f \equiv A \ [\gamma]^f \wedge B \ [\gamma]^f$$

- A konjunkció bevezető szabálya ($\wedge \text{in}$): Ha A és B külön-külön bizonyítható, akkor $A \wedge B$ is igaz.

$$\wedge \text{in} : \forall \{\Gamma \ A \ B\} \rightarrow \text{Pf } \Gamma A \rightarrow \text{Pf } \Gamma B \rightarrow \text{Pf } \Gamma (A \wedge B)$$

- Az konjunkció kivezető szabálya ($\wedge \text{out}_1$): Az $A \wedge B$ -ből következik A.
- Az konjunkció kivezető szabálya ($\wedge \text{out}_2$): Az $A \wedge B$ -ből következik B.

$$\wedge \text{out}_1 : \forall \{\Gamma \ A \ B\} \rightarrow \text{Pf } \Gamma (A \wedge B) \rightarrow \text{Pf } \Gamma A$$

$$\wedge \text{out}_2 : \forall \{\Gamma \ A \ B\} \rightarrow \text{Pf } \Gamma (A \wedge B) \rightarrow \text{Pf } \Gamma B$$

3.6.3. Diszjunkció ($\vee$)

Az $A \vee B$ jelentése, hogy legalább az egyik (A vagy B) igaz.

$$_ \vee _ : \forall \{\Gamma\} \rightarrow \text{For } \Gamma \rightarrow \text{For } \Gamma \rightarrow \text{For } \Gamma$$

$$\vee [] : \forall \{\Gamma \ A \ B \ \Delta\} \{\gamma : \text{Sub } \Delta \ \Gamma\} \rightarrow (A \vee B) \ [\ \gamma \]^f \equiv A \ [\ \gamma \]^f \vee B \ [\ \gamma \]^f$$

- A diszjunkció bevezető szabálya ($\vee \text{in}_1$): Ha A igaz, akkor $A \vee B$ is igaz.
- A diszjunkció bevezető szabálya ($\vee \text{in}_2$): Ha B igaz, akkor $A \vee B$ is igaz.

$$\vee \text{in}_1 : \forall \{\Gamma \ A \ B\} \rightarrow \text{Pf } \Gamma \ A \rightarrow \text{Pf } \Gamma \ (A \vee B)$$

$$\vee \text{in}_2 : \forall \{\Gamma \ A \ B\} \rightarrow \text{Pf } \Gamma \ B \rightarrow \text{Pf } \Gamma \ (A \vee B)$$

- A diszjunkció kivezető szabálya ($\vee \text{out}$): Ha tudjuk, hogy $A \vee B$ igaz, és abból is következik, ha A igaz, de abból is, ha B igaz, akkor C igaz.

$$\begin{aligned} \vee \text{out} : \forall \{\Gamma \ A \ B \ C\} \rightarrow \text{Pf } (\Gamma \triangleright_p A) \ (C \ [\ \text{p}_p \]^f) \rightarrow \text{Pf } (\Gamma \triangleright_p B) \ (C \ [\ \text{p}_p \]^f) \\ \rightarrow \text{Pf } \Gamma \ (A \vee B) \rightarrow \text{Pf } \Gamma \ C \end{aligned}$$

3.6.4. Konstans igaz ($\top$)

A $\top$ egy mindig igaz állítást jelöl.

$$\top : \forall \{\Gamma\} \rightarrow \text{For } \Gamma$$

$$\top [] : \forall \{\Gamma \ \Delta\} \{\gamma : \text{Sub } \Delta \ \Gamma\} \rightarrow \top \ [\ \gamma \]^f \equiv \top$$

- Az *igaz* bevezető szabálya ($\top \text{in}$): Minden kontextusban igaz.

$$\top \text{in} : \forall \{\Gamma\} \rightarrow \text{Pf } \Gamma \ \top$$

3.6.5. Konstans hamis ($\perp$)

Az $\perp$ egy ellentmondást vagy hamis állítást jelöl.

$$\perp : \forall \{\Gamma\} \rightarrow \text{For } \Gamma$$

$$\perp[] : \forall \{\Gamma \Delta\} \{\gamma : \text{Sub } \Delta \Gamma\} \rightarrow \perp [\gamma]^f \equiv \perp$$

- A *hamis* kivezető szabálya ($\perp\text{out}$): Ha egy bizonyítás során elérünk egy ellentmondást ($\perp$), akkor bármilyen tetszőleges állítást igaznak vehetünk (*ex falso quodlibet*).

$$\perp\text{out} : \forall \{\Gamma A\} \rightarrow \text{Pf } \Gamma \perp \rightarrow \text{Pf } \Gamma A$$

3.6.6. Kvantorok

A kvantorok formális logikai eszközök, amelyek segítségével általános és létezési állításokat tehetünk. Bindernek (kötés) is nevezzük őket, utalva rá, hogy egy változót köt, azaz meghatározza, hogy egy változó miként értelmezhető egy adott hatókörben.

Univerzális kvantor ($\forall$)

Az univerzális kvantor segítségével tudjuk kifejezni, hogy egy állítás minden lehetséges elemre igaz. Például a $\forall x P(x)$ azt jelenti, hogy minden x -re igaz, hogy $P(x)$. A deklarációban látható is, hogy mivel egy kötésről beszélünk, egy formulában lévő szabad változót kötünk meg, így egy szűkebb kontextusban lévő formulát fogunk kapni.

$$\text{Forall} : \forall \{\Gamma\} \rightarrow \text{For } (\Gamma \triangleright_t) \rightarrow \text{For } \Gamma$$

$$\begin{aligned} \text{Forall}[] : \forall \{\Gamma A \Delta\} \{\gamma : \text{Sub } \Delta \Gamma\} \\ \rightarrow \text{Forall } A [\gamma]^f \equiv \text{Forall } (A [\gamma \circ p_t \text{ ,t } q_t]^f) \end{aligned}$$

- Az univerzális kvantor bevezető szabálya ($\forall\text{in}$):

$$\forall\text{in} : \forall \{\Gamma A\} \rightarrow \text{Pf } (\Gamma \triangleright_t) A \rightarrow \text{Pf } \Gamma (\text{Forall } A)$$

- Az univerzális kvantor kivezető szabálya ($\forall\text{out}$):

$$\forall\text{out} : \forall \{\Gamma A\} \rightarrow \text{Pf } \Gamma (\text{Forall } A) \rightarrow \text{Pf } (\Gamma \triangleright_t) A$$

Egzisztenciális kvantor ($\exists$)

Az egzisztenciális kvantor azt fejezi ki, hogy létezik legalább egy olyan elem, amelyre igaz egy állítás. Például a $\exists x P(x)$ azt jelenti, hogy létezik legalább egy x , amelyre igaz, hogy $P(x)$.

$$\exists : \forall \{\Gamma\} \rightarrow \text{For } (\Gamma \triangleright_t) \rightarrow \text{For } \Gamma$$

$$\exists [] : \forall \{\Gamma \ A \ \Delta\} \{\gamma : \text{Sub } \Delta \ \Gamma\} \rightarrow \exists \ A \ [\ \gamma \]^f \equiv \exists \ (A \ [\ \gamma \circ p_t \ ,_t \ q_t \]^f)$$

- Az egzisztenciális kvantor bevezető szabálya ($\exists\text{in}$):

$$\exists\text{in} : \forall \{\Gamma \ A\} \rightarrow (t : \text{Tm } \Gamma) \rightarrow \text{Pf } \Gamma \ (A \ [\ \text{id} \ ,_t \ t \]^f) \rightarrow \text{Pf } \Gamma \ (\exists \ A)$$

- Az egzisztenciális kvantor kivezető szabálya ($\exists\text{out}$):

$$\exists\text{out} : \forall \{\Gamma \ A \ C\} \rightarrow \text{Pf } (\Gamma \triangleright_t \triangleright_p \ A) \ (C \ [\ p_t \circ p_p \]^f) \rightarrow \text{Pf } \Gamma \ (\exists \ A) \rightarrow \text{Pf } \Gamma \ C$$

4. fejezet

Szintaxis

A szintaxis az egyik specifikus algebrája az algebrai struktúrának, más néven iniciális algebrának is nevezzük. Az összes többi algebrát szemantikának nevezzük, erre láthatunk majd példát a következő fejezetben.

A szortokat algebrai adattípusokkal definiáljuk:

```
data Con : Set
data Sub : Con → Con → Set
data For : Con → Set
data Tm : Con → Set
```

A konstruktorokat megadjuk intuitívan a **Con** és **For** szortokhoz:

```
data Con where
  ◇      : Con
  _▷t    : Con → Con

data For where
  _⊃_    : ∀{Γ} → For Γ → For Γ → For Γ
  _∧_    : ∀{Γ} → For Γ → For Γ → For Γ
  ⊤      : ∀{Γ} → For Γ
  _∨_    : ∀{Γ} → For Γ → For Γ → For Γ
  ⊥      : ∀{Γ} → For Γ
  Forall : ∀{Γ} → For (Γ ▷t) → For Γ
  ∃      : ∀{Γ} → For (Γ ▷t) → For Γ
  Rel    : ∀{Γ}{n : ℕ} → relar n → Tm Γ ^ n → For Γ
```

Mivel a nyelvünk változókat is tartalmazhat, ezért be kell vezetnünk két további adattípust, a termváltozókat és a hozzá tartozó behelyettesítéseket:

```
data VTm : Con → Set where
  vz : ∀{Γ} → VTm (Γ ▷t)
  vs : ∀{Γ} → VTm Γ → VTm (Γ ▷t)

data VSub : Con → Con → Set where
  ε : ∀{Γ} → VSub Γ ◇
  _,v_ : ∀{Γ Δ} → VSub Δ Γ → VTm Δ → VSub Δ (Γ ▷t)
```

Az előbbiek segítségével már meg tudjuk adni a **Tm** és **Sub** szortokat is:

```
data Tm where
  var : ∀{Γ} → VTm Γ → Tm Γ
  fun : ∀{Γ}{n : ℕ} → funar n → Tm Γ ^ n → Tm Γ

data Sub where
  ε : ∀{Γ} → Sub Γ ◇
  _,t_ : ∀{Γ Δ} → Sub Δ Γ → Tm Δ → Sub Δ (Γ ▷t)
```

A bizonyítások miatt azonban szükségünk van még egy összetevőre, ez lesz a bizonyítások kontextusa:

```
data Conp : Con → Set where
  ◇p : ∀{Γ} → Conp Γ
  _,▷p_ : ∀{Γ} → Conp Γ → For Γ → Conp Γ
```

Továbbá szükségünk lesz még a **Conp** behelyettesítésére is, hogy a bizonyítások szortját meg tudjuk adni:

```
data Pf : (Γ : Con)(Γp : Conp Γ) → For Γ → Prop

data Subp : (Γ : Con) → Conp Γ → Conp Γ → Prop where
  idp : ∀{Γ Γp} → Subp Γ Γp Γp
  ε : ∀{Γ Γp} → Subp Γ Γp ◇p
  _,▷p_ : ∀{Δ Γp Δp A} → Subp Δ Δp Γp → Pf Δ Δp A → Subp Δ Δp (Γp ▷p A)
  pp : ∀{Γ Γp A} → Subp Γ (Γp ▷p A) (Γp [ idt ] Conp)
  _,◦p_ : ∀{Ξ Θp Δp Γp} → Subp Ξ Δp Γp → Subp Ξ Θp Δp → Subp Ξ Θp Γp
```

$$\begin{aligned} \llbracket _ \rrbracket^p &: \forall \{ \Xi \ \Psi \ \Delta_p \ \Gamma_p \} \rightarrow \text{Subp} \ \Xi \ \Delta_p \ \Gamma_p \rightarrow (\xi : \text{Sub} \ \Psi \ \Xi) \\ &\rightarrow \text{Subp} \ \Psi \ (\Delta_p \llbracket \xi \rrbracket^{\text{Comp}}) (\Gamma_p \llbracket \xi \rrbracket^{\text{Comp}}) \end{aligned}$$

data Pf where

$$\begin{aligned} \llbracket _ \rrbracket^p &: \forall \{ \Gamma \ \Gamma_p \ \Delta \ A \} \rightarrow \text{Pf} \ \Gamma \ \Gamma_p \ A \rightarrow (\gamma : \text{Sub} \ \Delta \ \Gamma) \\ &\rightarrow \text{Pf} \ \Delta \ (\Gamma_p \llbracket \gamma \rrbracket^{\text{Comp}}) (A \llbracket \gamma \rrbracket^f) \\ \llbracket _ \rrbracket^{pp} &: \forall \{ \Xi \ \Gamma_p \ \Delta_p \ A \} \rightarrow \text{Pf} \ \Xi \ \Gamma_p \ A \rightarrow (\gamma_p : \text{Subp} \ \Xi \ \Delta_p \ \Gamma_p) \rightarrow \text{Pf} \ \Xi \ \Delta_p \ A \\ \mathbf{q}_p &: \forall \{ \Gamma \ \Gamma_p \ A \} \rightarrow \text{Pf} \ \Gamma \ (\Gamma_p \triangleright_p A) \ A \\ \supset \text{in} &: \forall \{ \Gamma \ \Gamma_p \ A \ B \} \rightarrow \text{Pf} \ \Gamma \ (\Gamma_p \triangleright_p A) \ B \rightarrow \text{Pf} \ \Gamma \ \Gamma_p \ (A \supset B) \\ \supset \text{out} &: \forall \{ \Gamma \ \Gamma_p \ A \ B \} \rightarrow \text{Pf} \ \Gamma \ \Gamma_p \ (A \supset B) \rightarrow \text{Pf} \ \Gamma \ \Gamma_p \ A \rightarrow \text{Pf} \ \Gamma \ \Gamma_p \ B \\ \wedge \text{in} &: \forall \{ \Gamma \ \Gamma_p \ A \ B \} \rightarrow \text{Pf} \ \Gamma \ \Gamma_p \ A \rightarrow \text{Pf} \ \Gamma \ \Gamma_p \ B \rightarrow \text{Pf} \ \Gamma \ \Gamma_p \ (A \wedge B) \\ \wedge \text{out}_1 &: \forall \{ \Gamma \ \Gamma_p \ A \ B \} \rightarrow \text{Pf} \ \Gamma \ \Gamma_p \ (A \wedge B) \rightarrow \text{Pf} \ \Gamma \ \Gamma_p \ A \\ \wedge \text{out}_2 &: \forall \{ \Gamma \ \Gamma_p \ A \ B \} \rightarrow \text{Pf} \ \Gamma \ \Gamma_p \ (A \wedge B) \rightarrow \text{Pf} \ \Gamma \ \Gamma_p \ B \\ \top \text{in} &: \forall \{ \Gamma \ \Gamma_p \} \rightarrow \text{Pf} \ \Gamma \ \Gamma_p \ \top \\ \vee \text{in}_1 &: \forall \{ \Gamma \ \Gamma_p \ A \ B \} \rightarrow \text{Pf} \ \Gamma \ \Gamma_p \ A \rightarrow \text{Pf} \ \Gamma \ \Gamma_p \ (A \vee B) \\ \vee \text{in}_2 &: \forall \{ \Gamma \ \Gamma_p \ A \ B \} \rightarrow \text{Pf} \ \Gamma \ \Gamma_p \ B \rightarrow \text{Pf} \ \Gamma \ \Gamma_p \ (A \vee B) \\ \vee \text{out} &: \forall \{ \Gamma \ \Gamma_p \ A \ B \ C \} \rightarrow \text{Pf} \ \Gamma \ (\Gamma_p \triangleright_p A) \ (C) \rightarrow \text{Pf} \ \Gamma \ (\Gamma_p \triangleright_p B) \ (C) \\ &\rightarrow \text{Pf} \ \Gamma \ \Gamma_p \ (A \vee B) \rightarrow \text{Pf} \ \Gamma \ \Gamma_p \ C \\ \perp \text{out} &: \forall \{ \Gamma \ \Gamma_p \ A \} \rightarrow \text{Pf} \ \Gamma \ \Gamma_p \ \perp \rightarrow \text{Pf} \ \Gamma \ \Gamma_p \ A \\ \forall \text{in} &: \forall \{ \Gamma \ \Gamma_p \ A \} \rightarrow \text{Pf} \ (\Gamma \triangleright_t) (\Gamma_p \llbracket \mathbf{p}_t \rrbracket^{\text{Comp}}) A \rightarrow \text{Pf} \ \Gamma \ \Gamma_p \ (\text{Forall} \ A) \\ \forall \text{out} &: \forall \{ \Gamma \ \Gamma_p \ A \} \rightarrow \text{Pf} \ \Gamma \ \Gamma_p \ (\text{Forall} \ A) \rightarrow \text{Pf} \ (\Gamma \triangleright_t) (\Gamma_p \llbracket \mathbf{p}_t \rrbracket^{\text{Comp}}) A \\ \exists \text{in} &: \forall \{ \Gamma \ \Gamma_p \ A \} \rightarrow (t : \text{Tm} \ \Gamma) \rightarrow \text{Pf} \ \Gamma \ \Gamma_p \ (A \llbracket \text{id}_t \rrbracket_{\triangleright_t} t)^f \rightarrow \text{Pf} \ \Gamma \ \Gamma_p \ (\exists \ A) \\ \exists \text{out} &: \forall \{ \Gamma \ \Gamma_p \ A \ C \} \rightarrow \text{Pf} \ (\Gamma \triangleright_t) (\Gamma_p \llbracket \mathbf{p}_t \rrbracket^{\text{Comp}} \triangleright_p A) \ (C \llbracket \mathbf{p}_t \rrbracket^f) \\ &\rightarrow \text{Pf} \ \Gamma \ \Gamma_p \ (\exists \ A) \rightarrow \text{Pf} \ \Gamma \ \Gamma_p \ C \end{aligned}$$

Az iniciális modell tehát a következőképpen néz ki:

$$\begin{aligned} \mathbf{I} &: \text{Model} \\ \mathbf{I} &= \text{record} \\ &\{ \text{Con} = \Sigma \text{Con Comp} \\ &; \text{Sub} = \lambda (\Delta, \Delta_p) (\Gamma, \Gamma_p) \\ &\quad \rightarrow \Sigma (\text{Sub} \ \Delta \ \Gamma) \ \lambda \gamma \rightarrow \text{Lift} \ (\text{Subp} \ \Delta \ \Delta_p \ (\Gamma_p \llbracket \gamma \rrbracket^{\text{Comp}})) \\ &; _ \circ _ = \lambda \{ (\Gamma, \Gamma_p) (\Delta, \Delta_p) (\Theta, \Theta_p) \} (\gamma, \text{mk} \ \gamma_p) (\delta, \text{mk} \ \delta_p) \\ &\quad \rightarrow (\gamma \circ \delta), \text{mk} \ (\text{substP} \ (\text{Subp} \ \Theta \ \Theta_p) \ ([\circ] \text{Comp}^{-1}) ((\gamma_p \llbracket \delta \rrbracket^p) \circ_p \delta_p)) \\ &; \text{ass} = \text{cong, ass} \\ &; \text{id} = \lambda \{ (\Gamma, \Gamma_p) \} \rightarrow \text{id}_t, \text{mk} \ (\text{substP} \ (\lambda x \rightarrow \text{Subp} \ \Gamma \ \Gamma_p \ x) \ ([\text{id}] \text{Comp}^{-1}) \ \text{idp}) \end{aligned}$$

```

; idl = cong, idl
; idr = cong, idr
; ◇ = ◇ , ◇p
; ε = ε , mk ε
; ◇η = cong (▬, mk ▬) ◇η
; Tm = λ (Γ , Γp) → Tm Γ
; ▬[▬]t = λ t (s , sp) → t [ s ]t
; [o]t = λ {Γ Δ Θ t (γ , γp) (δ , δp)} → [o]t {t = t}
; [id]t = t[id]t
; ▬▷t = λ (Γ , Γp) → Γ ▷t , Γp [ pt ]Conp
; ▬, t ▬ = λ (γ , mk γp) t
      → (γ , t t) ,
      mk (substP (Subp ▬ ▬) (cong (▬ [▬]Conp) (▷tβ1-1) □ [o]Conp) γp)
; pt = pt , mk idp
; qt = qt
; ▷tβ1 = cong, ▷tβ1
; ▷tβ2 = refl
; ▷tη = cong, ▷tη
; For = λ (Γ , Γp) → For Γ
; ▬[▬]f = λ A (s , sp) → A [ s ]f
; [o]f = [o]f
; [id]f = f[id]f
; Pf = λ (Γ , Γp) A → Pf Γ Γp A
; ▬[▬]p = λ { {Γ , Γp} {Δ , Δp} {A} } a (γ , mk γp) → (a [ γ ]p) [ γp ]pp }
; ▬▷p▬ = λ (Γ , Γp) A → Γ , Γp ▷p A
; ▬, p ▬ = λ (s , (mk sp)) p → s , mk (sp , p p)
; pp = idt , mk pp
; qp = λ { (Γ , Γp) } {A} → substP (λ x → Pf Γ (Γp ▷p A) x) (f[id]f-1) qp
; ▷pβ1 = cong, idl
; ▷pη = cong, idl-1
; Rel = Rel
; Rel[] = λ {Γ n ar} → cong (λ x → Rel ar x) s≡map
; fun = fun
; fun[] = λ {Γ n ar} → cong (λ x → fun ar x) s≡map

```

```

;  $\_ \supset \_ = \_ \supset \_$ 
;  $\supset [] = \text{refl}$ 
;  $\supset \text{in} = \lambda \{(\Gamma, \Gamma_p) A B\} \rightarrow \text{substP } (\lambda x \rightarrow \text{Pf } \Gamma (\Gamma_p \triangleright_p A) x$ 
     $\rightarrow \text{Pf } \Gamma \Gamma_p (A \supset B)) (\text{f[id]}^f)^{-1} (\supset \text{in})$ 
;  $\supset \text{out} = \supset \text{out}$ 
;  $\_ \wedge \_ = \_ \wedge \_$ 
;  $\wedge [] = \text{refl}$ 
;  $\wedge \text{in} = \wedge \text{in}$ 
;  $\wedge \text{out}_1 = \wedge \text{out}_1$ 
;  $\wedge \text{out}_2 = \wedge \text{out}_2$ 
;  $\top = \top$ 
;  $\top [] = \text{refl}$ 
;  $\top \text{in} = \top \text{in}$ 
;  $\_ \vee \_ = \_ \vee \_$ 
;  $\vee [] = \text{refl}$ 
;  $\vee \text{in}_1 = \vee \text{in}_1$ 
;  $\vee \text{in}_2 = \vee \text{in}_2$ 
;  $\vee \text{out} = \lambda \{(\Gamma, \Gamma_p)\} \{A\} \{B\} \{C\} a b$ 
     $\rightarrow \text{substP } (\lambda C \rightarrow \text{Pf } \Gamma \Gamma_p (A \vee B) \rightarrow \text{Pf } \Gamma \Gamma_p C)$ 
     $\text{f[id]}^f (\vee \text{out } a b)$ 
;  $\perp = \perp$ 
;  $\perp [] = \text{refl}$ 
;  $\perp \text{out} = \perp \text{out}$ 
;  $\text{Forall} = \text{Forall}$ 
;  $\text{Forall} [] = \text{cong } (\lambda z \rightarrow \text{Forall } (\_ [z, \text{t} \text{ var } \text{vz}]^f)) (\text{op}^{-1})$ 
;  $\forall \text{in} = \forall \text{in}$ 
;  $\forall \text{out} = \forall \text{out}$ 
;  $\exists = \exists$ 
;  $\exists [] = \text{cong } (\lambda z \rightarrow \exists (\_ [z, \text{t} \text{ var } \text{vz}]^f)) (\text{op}^{-1})$ 
;  $\exists \text{in} = \exists \text{in}$ 
;  $\exists \text{out} = \lambda \{(\Gamma, \Gamma_p)\} \{A\} \{C\} w p$ 
     $\rightarrow \exists \text{out } \{\Gamma\} \{\Gamma_p\} \{A\} \{C\}$ 
     $(\text{substP } (\lambda z$ 
     $\rightarrow \text{Pf } (\Gamma \triangleright_t) ((\Gamma_p [p_t] \text{Conp}) \triangleright_p A) (C [z]^f)) \text{idr } w) p$ 

```

}

5. fejezet

Modellek

A szemantikai modellek a megadott elsőrendű logika jelentését, értelmezését adják meg. A három vizsgált modell - a Bool modell, a Tarski modell és a Family modell - különböző megközelítéseket kínál a logikai kifejezések értelmezésére és kiértékelésére.

- Bool modell: A legegyszerűbb és legközvetlenebb megközelítés, amely a logikai kifejezéseket az igaz és hamis értékekre képezi le, valamint a logikai műveletek a Bool értékek közötti műveletekkel azonosak. Ez a klasszikus kétértékű szemantika.
- Tarski modell: A logikai kifejezések értelmezését a halmazelmélet alapjaira helyezi. Ebben a modellben a logikai változók és kifejezések értékei halmazok, illetve halmazok közötti relációk. A Tarski szemantika a logikai igazságot a kifejezések halmazelméleti interpretációjával határozza meg, amely lehetővé teszi a logikai állítások formális és precíz vizsgálatát.
- Family modell: Egy indexelt családok segítségével felépített szemantika. Felfogható a Kripke-szemantika egy specializációjaként. Ez a modell, a Kripke modellel szemben, nem specifikál explicit elérhetőségi relációt az indexek között, ami egyszerűsíti a struktúrát.

5.1. Bool modell

A Bool modellhez közvetlenül az Agda beépített Bool típusát használjuk, így a logikai összekötők ($\wedge$, $\vee$, $\rightarrow$, $\neg$) is a megfelelő Bool műveletekre fordítódnak. A

formulák kiértékelése során minden atomi állítás, predikátum és függvény végső soron egy Bool értékre redukálódik. Az univerzális és egzisztenciális kvantorokra gondolhatunk úgy, mint listák feletti $\forall$ és $\exists$ műveletek.

5.2. Tarski modell

A Tarski modell az elsőrendű logika standard szemantikai modellje. Egy nem üres domain (univerzum) felett értelmezett interpretáció.

Az Agdában implementált Tarski modell a következő komponensekből áll:

Egy tetszőleges típus, amely a domaint reprezentálja
Függvények, amelyek a logikai nyelv szimbólumait a domain elemeire, függvényeire és relációira képezik le
Változókiértékelő függvény, amely a változókat a domain elemeire képezi le
Rekurzív interpretációs függvény, amely a formulákat a domain feletti állításokra képezi le

5.3. Family modell

A Family modell felfogható a Kripke-szemantika egy specializációjaként.

Indexelés mint lehetséges világok: A Family modellben szereplő I indexhalmaz elemei megfeleltethetők a Kripke szemantika lehetséges világainak. Az elérhetőségi reláció implicit módon adott vagy triviális A $D(i)$ tartományok explicit módon specifikáltak minden i indexre A logikai operátorok szemantikája hasonlóan van definiálva, de a modell szerkezete egyszerűbb és célorientáltabb

Ahogy a Kripke-modellekben minden lehetséges világban kiértékeljük a formulákat, a Family modellben minden indexhez tartozó struktúrán értelmezzük a kifejezéseket és formulákat. Elérhetőségi reláció hiánya: A Kripke-modellekben alapvető szerepet játszik a világok közötti elérhetőségi reláció (R), amely meghatározza a modális operátorok szemantikáját. A Family modell nem specifikál explicit elérhetőségi relációt az indexek között, ami egyszerűsíti a struktúrát.

Minden predikátum egy típuscsalád, amely a domain elemeihez típusokat rendel A logikai konnektívák típuskonstruktorokként implementálódnak (pl. függő összeg és szorzat típusok) A kvantorok függő típusokként implementálódnak (Π és Σ típusok) A bizonyítások a megfelelő típusok értékei/lakói

5.4. Modellek összehasonlítása

A modellek összehasonlítása

Expresszivitás

Bool modell: A legkevésbé expresszív, csak véges domainek felett használható hatékonyan, és a klasszikus kétértékű logikára korlátozódik. Tarski modell: Nagyon expresszív, a klasszikus elsőrendű logika teljes szemantikáját képes modellezni, beleértve a végtelen domaineket is. Family modell: Expresszív és konstruktív jellegű, de más filozófiai alapokon nyugszik, mint a klasszikus logika.

Implementációs komplexitás

Bool modell: Egyszerű implementáció, könnyen érthető kód. Tarski modell:

Közepes komplexitás, több absztrakciós réteget igényel. Family modell: A legösszetettebb implementáció, amely mély típuselméleti ismereteket igényel.

Alkalmazási területek

Bool modell: Ideális tautológiák ellenőrzésére, véges modell ellenőrzésre és egyszerű példákra. Tarski modell: Alkalmas általános célú logikai modellezésre, tételbizonyítókhoz és modellellenőrzőkhöz. Family modell: Kiválóan alkalmas bizonyítássegédkekhez, programlevezetéshez és verifikációhoz.

5.5. Összegzés

A három bemutatott modell különböző megközelítéseket kínál az elsőrendű logika szemantikájának implementálására Agdában. Mindegyik modellnek megvannak a maga erősségei és gyengeségei, valamint jellemző alkalmazási területei:

A Bool modell egyszerűsége és hatékonysága miatt alkalmas gyors prototípusok készítésére és véges domainek feletti ellenőrzésekre. A Tarski modell matematikai precizitása és általánossága miatt a legjobb választás a klasszikus elsőrendű logika teljes szemantikájának implementálására. A Family modell típuselméleti megalapozottsága miatt különösen alkalmas olyan alkalmazásokhoz, ahol a bizonyítások és számítások integrációja fontos.

Az implementációnkban mindhárom modellt megvalósítottuk, demonstrálva ezzel az Agda típusrendszerének erejét és rugalmasságát. A modellek közötti formális kapcsolatokat is definiáltuk, bizonyítva, hogy bizonyos feltételek mellett konzisztens eredményeket adnak.

6. fejezet

Összegzés

Related work

model.lagda be-
helyettesítési
szabályok ma-
gyarázat